Universidade de Brasília

Organização e Arquitetura de Computadores - Turma 3

Professor: Ricardo Pezzuol Jacobi

Processador RISC-V Uniciclo

Autor:

Leonardo Pereira Côrtes - 20/0030582

Brasília

11 de Dezembro de 2023

Sumário

Sumário.....2
Introdução.....3
Implementação.....4
Testes.....5
Conclusão.....7

Introdução

O projeto da disciplina consiste no desenvolvimento de uma versão do processador RISC-V Uniciclo em VHDL. A plataforma de desenvolvimento utilizada para o desenvolvimento do projeto foi o ModelSim-Altera.

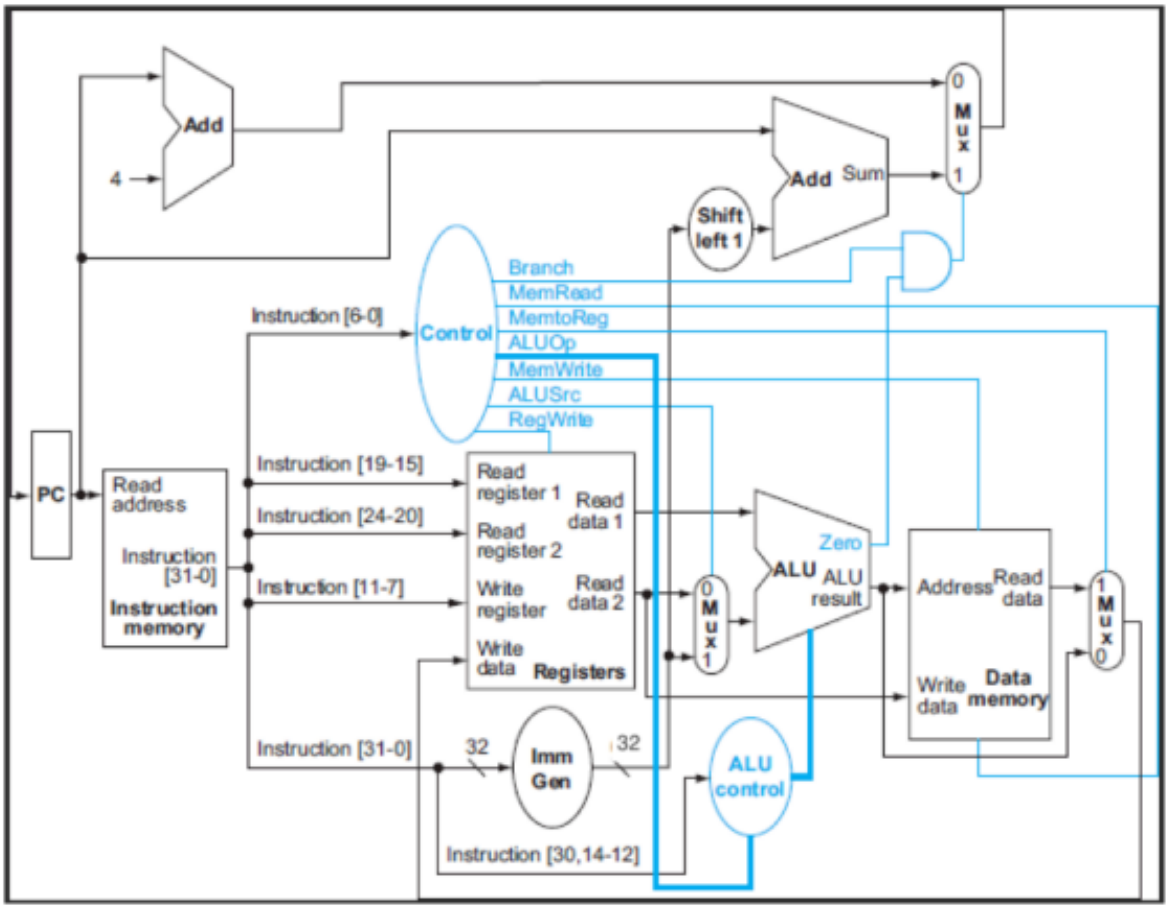

Figura 1. RISC-V Uniciclo.

Foi utilizado como base o diagrama da Figura 1, porém ele está incompleto, para implementação das instruções LUI, AUIPC, JAL e JALR, o diagrama foi modificado para o da Figura 2. Foram adicionados 4 sinais de controle e 4 multiplexadores .

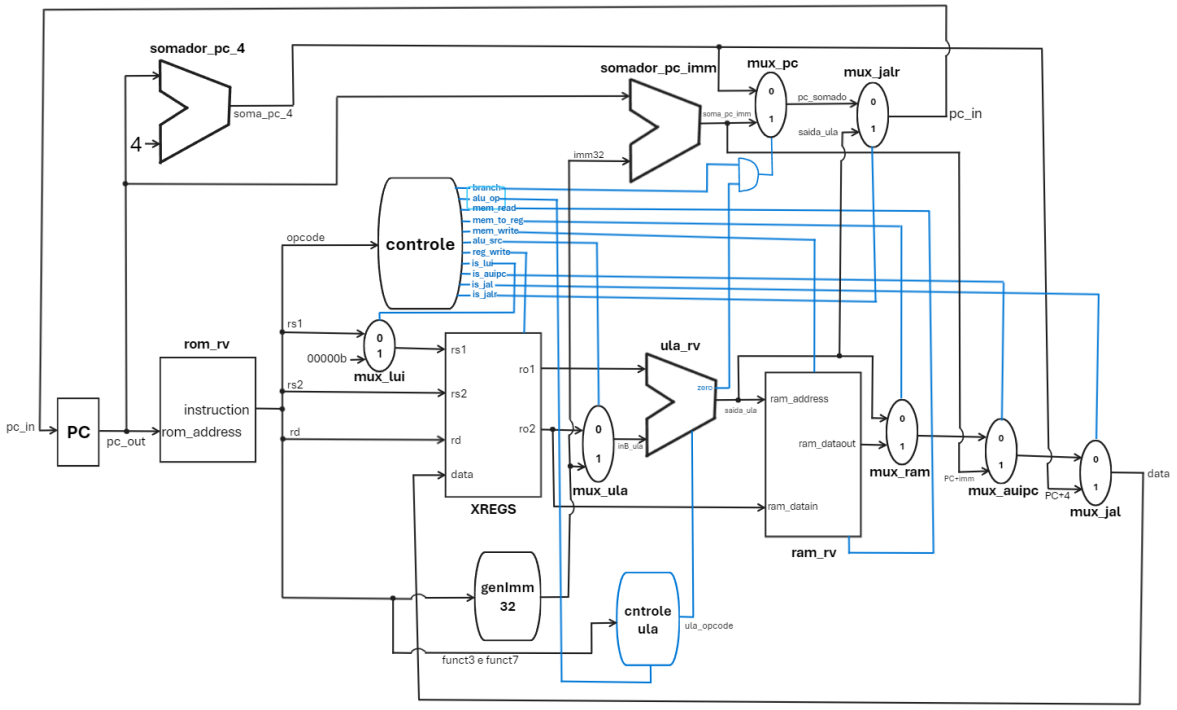

Figura 2. RISC-V Uniciclo modificado.

Implementação

Boa parte dos módulos já haviam sido implementados durante os trabalhos da disciplina (Gerador de Imediatos, Memória de Instruções - ROM, Banco de Registradores - XREGS, Unidade Lógica Aritmética - ULA e Memória de Dados - RAM), para o projeto final foi implementado adicionalmente:

- **PC**: contador de programa. É um registrador de 32 bits. Entretanto, pelas restrições de memória adotadas (memórias com 8 bits de endereço), foram enviados a memória de instruções apenas os bits 9 a 2, sendo o restante ignorado;
- **Módulo de Controle**: recebe o opcode da memória de instruções que faz a escolha entre os sinais: *branch*, *mem_read*, *mem_to_reg*, *mem_write*, *alu_src*, *reg_write*, *is_lui*, *is_auipc*, *is_jal* e *is_jalr*;
- **Módulo de Controle da ULA**: recebe o sinal de controle *alu_op*, e os valores de *funct3* e *funct7*, e retorna o *opcode* da ULA em 4 bits.
- **Multiplexadores**: foram implementados multiplexadores 2 pra 1 com saída e entrada de 32 bits e um multiplexador com saída e entrada de 5 bits para instruções *lui*.
- **Somadores**: módulo de somador de 32 bits para operar com endereços.

- **Package *riscv_pkg***: módulo para iniciar os componentes e definir constantes para auxiliar operações.
- **Módulo *rv_uniciclo***: implementado para criar os sinais necessários, atribuir as portas aos módulos interligando-os, e por fim realizar as batidas do clock para rodar o programa.

Obs: os arquivos *ram_rv.vhd* e *rom_rv.vhd*, usam o padrão VHDL-2008 para manipular arquivos de texto.

Testes

Os testes foram realizados com um arquivo gerado pelo RARS disponibilizado pelo professor (*uniciclo.asm*). O arquivo utiliza o modelo de memória compacto e depois de rodá-lo foram salvos em arquivos de texto em hexadecimal os valores das memórias de instruções e de dados (*tests_text.txt* e *tests_data.txt*).

Os módulos da memória de instruções e de dados abrem esses arquivos de dados. A partir de um clock rodando a cada 1 ns o código é executado e podemos analisar os testes por meio das formas de onda.

Nas imagens a seguir podemos alguns valores relevantes: as batidas do clock (*aux_clk*), a saída do pc (*pc_out*), a saída da memória de instruções (*instruction*), a saída da ULA (*saida_ula*) e a saída da memória de dados (*ram_dataout*), as 10 imagens são referentes execução de todas as instruções do código fornecido.

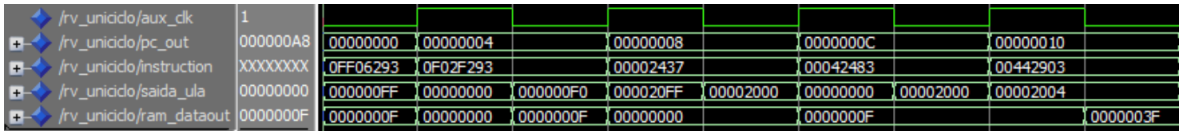

Figura 3. Instruções ORI, ANDI, LUI, LW e LW

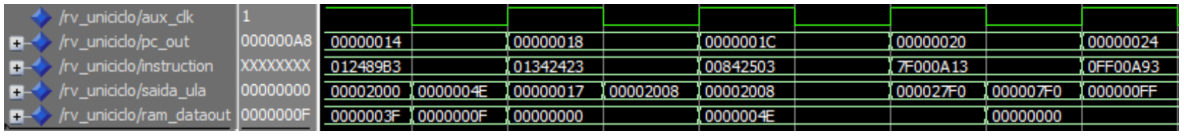

Figura 4. Instruções ADD, SW, LW, ADDI e ADDI

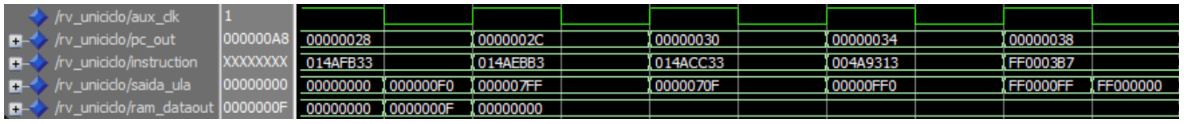

Figura 5. Instruções AND, OR, XOR, SLLI e LUI

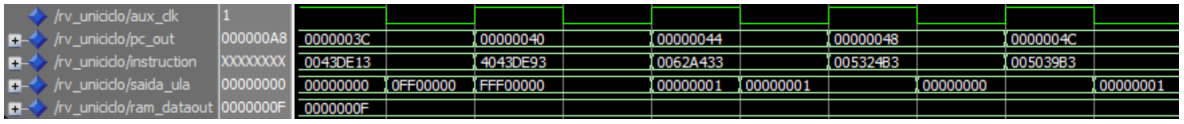

Figura 6. Instruções SRLI, SRAI, SLT, SLT, SLTU

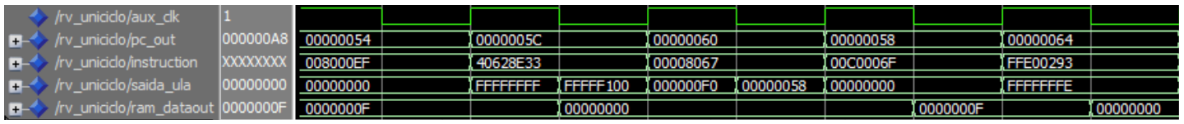

Figura 7. Instruções JAL, SUB, JALR, SLTU e ADDI

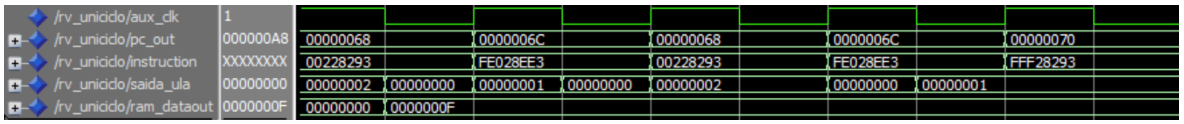

Figura 8. Instruções ADDI, BEQ, ADDI, BEQ e ADDI

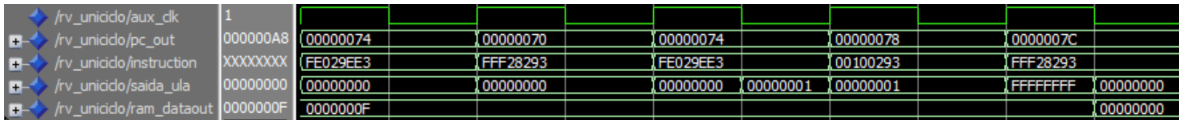

Figura 9. Instruções BNE, ADDI, BNE, ADDI e ADDI

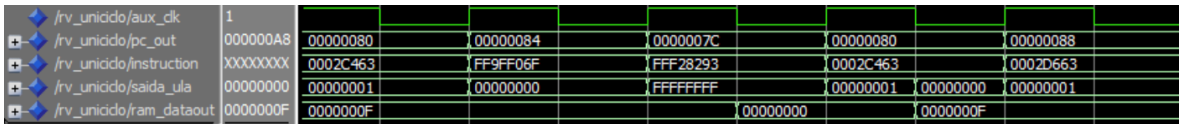

Figura 10. Instruções BLT, J, ADDI, BLT e BGE

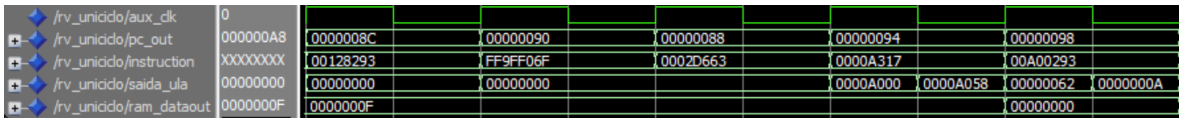

Figura 11. Instruções ADDI, J, BGE, AUIPC e ADDI

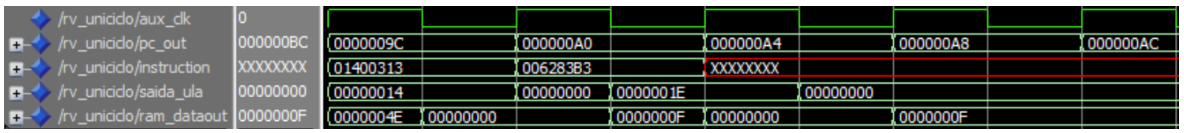

Figura 12. Instruções ADDI, ADD e fim do programa

Em alguns casos o sinal de saída da memória de dados (*ram_dataout*) não é igual ao resultado apresentado pelo RARS. Isso ocorre visto que nem todas as instruções acessam a memória, porém não afeta o resultado final do programa e não é um problema para o processador.

Conclusão

Foi possível desenvolver uma versão do processador RISC-V Uniciclo em VHDL na plataforma ModelSim-Altera, executando um código gerado pelo RARS utilizando o modelo de memória compacta. Percebe-se nos testes que os resultados obtidos foram de acordo com o esperado, comprovando a funcionalidade dos códigos desenvolvidos para cada módulo do processador e o funcionamento deles trabalhando em conjunto.